

HelixCode Architecture & System Design

System Architecture Overview

Version: 1.0.0 Date: 2026-04-30 Status: Foundation verified, critical bluff areas identified

1. High-Level Architecture

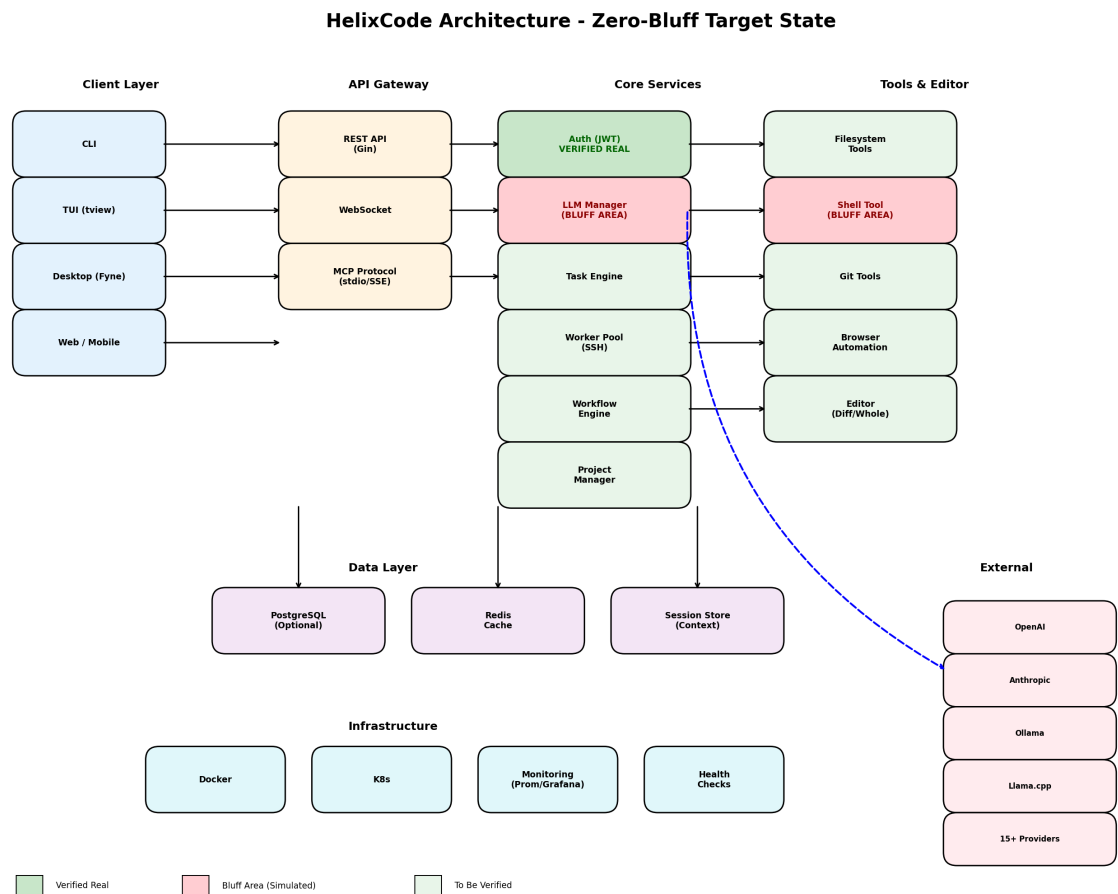


Figure 1: HelixCode Architecture

Architecture Layers

Layer	Components	Status
Client Layer	CLI, TUI (tview), Desktop (Fyne), Web/ Mobile	Structure exists, need verification
API Gateway	REST (Gin), WebSocket, MCP Protocol	Partially implemented
Core Services	Auth, LLM, Task, Worker, Workflow, Project	Auth VERIFIED REAL, LLM BLUFF AREA
Tools & Editor	Filesystem, Shell, Git, Browser, Editor	Need verification
Data Layer	PostgreSQL, Redis, Session Store	Schema advertised, need verification

Table 1 – continued

Layer	Components	Status
Infrastructure	Docker, K8s, Monitoring, Health Checks	Docker exists, need verification
External	15+ LLM Providers (OpenAI, Anthropic, Ollama, etc.)	NOT connected (simulated)

2. Component Interaction Flow

2.1 LLM Generation Flow (TARGET - Current is Simulated)

```

User -> CLI --prompt "Hello"
|
v
CLI.handleGenerate()
|
v
ProviderManager.Generate(ctx, req)
|
+---> Fallback Chain:
    1. OllamaProvider.Generate() --> HTTP POST /api/generate
    2. OpenAIProvider.Generate() --> HTTP POST /v1/chat/completions
    3. AnthropicProvider.Generate() --> HTTP POST /v1/messages
    |
    +---> Circuit Breaker (per provider)
        |
        +---> Real HTTP Call
            |
            +---> Response
                |
                v
            User receives actual AI-generated text

```

Current State (BLUFF): The flow stops at `CLI.handleGenerate()` which returns a simulated response without ever calling `ProviderManager`.

Required Fix: Implement real provider dispatch with circuit breakers and fallback chains.

2.2 Task Distribution Flow (TARGET)

```

User -> API: POST /api/v1/tasks (project build)
|
v
TaskManager.Create(ctx, task)
|
v
TaskManager.Distribute(task)
|
+---> Worker Pool Selection:
    - Filter by capabilities (CPU, GPU, memory)

```

```

- Health check (SSH probe)
- Load balancing (least loaded)
|
+---> SSHWorkerPool.Dispatch(task, worker)
    |
    +---> SSH Connection
        |
        +---> Remote Execution
            |
            +---> Checkpoint every 300s
                |
                +---> Result Collection
                    |
                    v
                    TaskManager.Complete(task, result)

```

2.3 Authentication Flow (VERIFIED REAL)

```

User -> API: POST /api/v1/auth/register
|
v
AuthService.Register(username, email, password)
|
+---> validateRegistration() [input validation]
|
+---> hashPassword() [bcrypt with configured cost]
|
+---> db.CreateUser() [persist to PostgreSQL]
|
v
User created

User -> API: POST /api/v1/auth/login
|
v
AuthService.Login(username, password)
|
+---> db.GetUserByUsername() [fetch user + hash]
|
+---> verifyPassword() [bcrypt.CompareHashAndPassword]
    |
    +---> fallback: verifyArgon2Password()
        [parse Argon2 params, decode salt, compute hash, constant-time compare]
|
+---> generateSessionToken() [crypto/rand, 32 bytes]
|
+---> db.CreateSession() [persist session]
|
v
Session returned with JWT token

```

```
User -> API: GET /api/v1/workers (with Bearer token)
|
v
AuthService.VerifyJWT(token)
|
+---> jwt.Parse() [validate signature, check expiry]
|
+---> VerifyJWTWithDB() [optional: fetch complete user, check IsActive]
|
v
Request authorized
```

Verified Quality: This flow uses real cryptography (bcrypt/argon2), real database operations, proper input validation, and secure token generation. This is the quality standard all other subsystems must match.

3. Database Schema (Advertised - Verification Required)

Core Tables

```
-- users: User accounts and authentication
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username VARCHAR(255) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  display_name VARCHAR(255),
  is_active BOOLEAN DEFAULT true,
  is_verified BOOLEAN DEFAULT false,
  mfa_enabled BOOLEAN DEFAULT false,
  last_login TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- sessions: Active user sessions
CREATE TABLE sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  session_token VARCHAR(255) UNIQUE NOT NULL,
  client_type VARCHAR(50),
  ip_address INET,
  user_agent TEXT,
  expires_at TIMESTAMP NOT NULL,
  created_at TIMESTAMP DEFAULT NOW()
);

-- workers: Distributed worker nodes
CREATE TABLE workers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

hostname VARCHAR(255) NOT NULL,
display_name VARCHAR(255),
ssh_host VARCHAR(255),
ssh_port INTEGER DEFAULT 22,
ssh_username VARCHAR(255),
ssh_key_path VARCHAR(255),
status VARCHAR(50) DEFAULT 'inactive',
cpu_count INTEGER,
memory_bytes BIGINT,
gpu_count INTEGER,
gpu_model VARCHAR(255),
last_heartbeat TIMESTAMP,
created_at TIMESTAMP DEFAULT NOW()
);

-- tasks: Task management with checkpoints
CREATE TABLE tasks (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID,
    title VARCHAR(255) NOT NULL,
    description TEXT,
    status VARCHAR(50) DEFAULT 'pending',
    priority INTEGER DEFAULT 0,
    worker_id UUID REFERENCES workers(id),
    dependencies UUID[],
    checkpoint_data JSONB,
    result_data JSONB,
    retry_count INTEGER DEFAULT 0,
    max_retries INTEGER DEFAULT 3,
    started_at TIMESTAMP,
    completed_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT NOW()
);

-- projects: Project lifecycle
CREATE TABLE projects (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    description TEXT,
    status VARCHAR(50) DEFAULT 'planning',
    owner_id UUID REFERENCES users(id),
    config JSONB,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);

-- llm_providers: Provider configurations
CREATE TABLE llm_providers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    provider_type VARCHAR(50) NOT NULL,

```

```
config JSONB,
is_enabled BOOLEAN DEFAULT true,
priority INTEGER DEFAULT 0,
last_health_check TIMESTAMP,
created_at TIMESTAMP DEFAULT NOW()
);

-- notifications: Notification history
CREATE TABLE notifications (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title VARCHAR(255),
  message TEXT,
  type VARCHAR(50),
  priority VARCHAR(50),
  channels VARCHAR(50)[],
  status VARCHAR(50) DEFAULT 'pending',
  sent_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW()
);
```

Verification Required: These tables must exist in migration files and be verified by component tests.

4. API Specification

4.1 Authentication Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/v1/auth/register	User registration	No
POST	/api/v1/auth/login	User login	No
POST	/api/v1/auth/refresh	Token refresh	Yes
POST	/api/v1/auth/logout	Logout	Yes

4.2 Worker Endpoints

Method	Endpoint	Description	Auth Required
GET	/api/v1/workers	List workers	Yes
POST	/api/v1/workers	Register worker	Yes
GET	/api/v1/workers/:id	Get worker details	Yes
POST	/api/v1/workers/:id/health	Worker health check	Yes

4.3 Task Endpoints

Method	Endpoint	Description	Auth Required
GET	/api/v1/tasks	List tasks	Yes

Table 4 – continued

Method	Endpoint	Description	Auth Required
POST	/api/v1/tasks	Create task	Yes
GET	/api/v1/tasks/:id	Get task details	Yes
POST	/api/v1/tasks/:id/ start	Start task	Yes
POST	/api/v1/tasks/:id/ checkpoint	Create checkpoint	Yes

4.4 LLM Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/v1/llm/generate	Generate text	Yes
POST	/api/v1/llm/generate/ stream	Stream generation	Yes
GET	/api/v1/llm/models	List models	Yes
GET	/api/v1/llm/providers	List providers	Yes
POST	/api/v1/llm/ providers/:id/health	Provider health	Yes

Anti-Bluff Requirement: The /api/v1/llm/generate endpoint MUST make real HTTP calls to configured providers. The response MUST NOT contain simulated content.

5. Provider Architecture

5.1 Provider Interface

```
type Provider interface {
    // Generate completes a single generation request
    Generate(ctx context.Context, req *GenerateRequest) (*GenerateResponse, error)

    // GenerateStream returns a channel of response chunks
    GenerateStream(ctx context.Context, req *GenerateRequest) (<-chan *GenerateChunk, error)

    // GetModels returns available models from this provider
    GetModels() ([]Model, error)

    // GetCapabilities returns provider capabilities
    GetCapabilities() *Capabilities

    // ValidateConfig validates provider-specific configuration
    ValidateConfig(config map[string]interface{}) error

    // HealthCheck verifies provider is accessible
    HealthCheck(ctx context.Context) error
}
```

5.2 Provider Registry

```
type ProviderRegistry struct {
    providers map[string]Provider
    mu        sync.RWMutex
}

func (r *ProviderRegistry) Register(name string, provider Provider) {
    r.mu.Lock()
    defer r.mu.Unlock()
    r.providers[name] = provider
}

func (r *ProviderRegistry) Get(name string) (Provider, bool) {
    r.mu.RLock()
    defer r.mu.RUnlock()
    p, ok := r.providers[name]
    return p, ok
}
```

5.3 Provider Implementations Required

Provider	Protocol	Status
Ollama	HTTP /api/generate	TO IMPLEMENT
Llama.cpp	HTTP /completion	TO IMPLEMENT
OpenAI	HTTP /v1/chat/completions	TO IMPLEMENT
Anthropic	HTTP /v1/messages	TO IMPLEMENT
Gemini	HTTP /v1beta/models	TO IMPLEMENT
xAI/Grok	HTTP /v1/chat/completions	TO IMPLEMENT
OpenRouter	HTTP /v1/chat/completions	TO IMPLEMENT
GitHub Copilot	HTTP /v1/engines/copilot-codex/ completions	TO IMPLEMENT
Qwen	HTTP /compatible-mode/v1/chat/ completions	TO IMPLEMENT
Azure	HTTP /openai/deployments	TO IMPLEMENT
AWS Bedrock	AWS SDK	TO IMPLEMENT
VertexAI	HTTP /v1/projects	TO IMPLEMENT
Groq	HTTP /v1/chat/completions	TO IMPLEMENT
vLLM	HTTP /v1/completions	TO IMPLEMENT
KoboldAI	HTTP /api/v1/generate	TO IMPLEMENT

6. Worker Architecture

6.1 SSH Worker Pool

```
type SSHWorkerPool struct {
    workers map[string]*SSHWorker
}
```

```

    mu          sync.RWMutex
    healthInterval time.Duration
    maxConcurrent int
}

type SSHWorker struct {
    Hostname      string
    DisplayName   string
    SSHConfig     *ssh.ClientConfig
    Status        WorkerStatus
    Capabilities  *WorkerCapabilities
    LastHeartbeat time.Time
}

```

6.2 Worker Lifecycle

1. **Registration:** Admin adds worker via CLI or API
2. **Connection:** Pool establishes SSH connection
3. **Capability Discovery:** Worker reports CPU, GPU, memory
4. **Health Monitoring:** Periodic SSH probe every 30s
5. **Task Dispatch:** Tasks assigned based on capabilities
6. **Auto-Installation:** If helix binary missing, auto-install via SSH
7. **Removal:** On failed health checks or admin command

7. Security Architecture

7.1 Authentication Layers

Layer	Mechanism	Implementation
API Auth	JWT Bearer tokens	VERIFIED REAL
Session	Redis-backed sessions	To verify
Worker	SSH key-based	To verify
Database	Password + SSL	To configure

7.2 Input Validation

- Path validation: Prevent directory traversal (../)
- Command blacklist: Prevent dangerous shell commands
- Schema validation: All tools validate params against JSON schema
- SQL injection: Use parameterized queries (pgx)

7.3 Sandboxing

- Shell execution: Whitelist + timeout + resource limits
- File operations: Restrict to workspace directory
- Network: Rate limiting on web requests
- Container: Non-root user, read-only root fs where possible

8. Testing Architecture

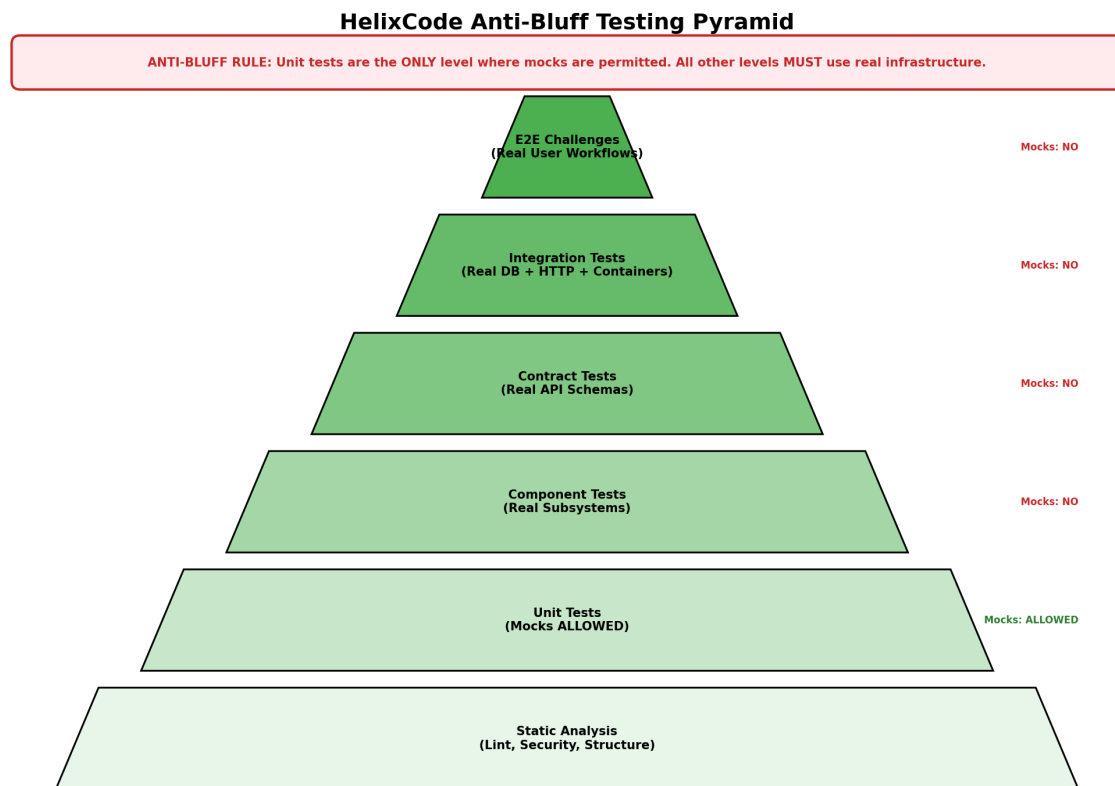


Figure 2: Testing Pyramid

8.1 Test Infrastructure

```
tests/
├─ unit/           # Unit tests (mocks OK)
│  ├─ auth_test.go
│  ├─ llm_test.go
│  └─ ...
├─ contract/       # Provider API contracts (NO MOCKS)
│  ├─ ollama_contract_test.go
│  └─ openai_contract_test.go
├─ component/      # Real subsystems (NO MOCKS)
│  ├─ database_component_test.go
│  └─ worker_component_test.go
├─ integration/    # Full app (NO MOCKS)
│  ├─ api_integration_test.go
│  └─ workflow_integration_test.go
├─ e2e/
│  └─ challenges/  # Real user workflows (NO MOCKS)
│     ├─ run_all_challenges.sh
│     ├─ llm_generation_challenge.sh
│     └─ command_execution_challenge.sh
```

```
|      └─ ...  
└─ security/      # OWASP tests (NO MOCKS)  
└─ performance/  # Benchmarks
```

9. Container Architecture

9.1 Docker Compose Stack

```
version: '3.8'  
services:  
  helixcode-server:  
    build: .  
    ports:  
      - "8080:8080"  
      - "2222:2222"  
    environment:  
      - HELIX_DATABASE_URL=postgres://helix:helixpass@postgres:5432/helixcode_prod  
      - HELIX_REDIS_URL=redis://redis:6379  
      - HELIX_AUTH_JWT_SECRET=${HELIX_AUTH_JWT_SECRET}  
    depends_on:  
      - postgres  
      - redis  
    healthcheck:  
      test: ["CMD", "curl", "-f", "http://localhost:8080/health"]  
      interval: 30s  
      timeout: 10s  
      retries: 3  
    networks:  
      - helix-network  
  
  postgres:  
    image: postgres:15-alpine  
    environment:  
      - POSTGRES_USER=helix  
      - POSTGRES_PASSWORD=${HELIX_DATABASE_PASSWORD}  
      - POSTGRES_DB=helixcode_prod  
    volumes:  
      - postgres_data:/var/lib/postgresql/data  
    healthcheck:  
      test: ["CMD-SHELL", "pg_isready -U helix"]  
      interval: 10s  
      timeout: 5s  
      retries: 5  
    networks:  
      - helix-network  
  
  redis:  
    image: redis:7-alpine  
    command: redis-server --requirepass ${HELIX_REDIS_PASSWORD}
```

```
volumes:
  - redis_data:/data
healthcheck:
  test: ["CMD", "redis-cli", "ping"]
  interval: 10s
  timeout: 5s
  retries: 5
networks:
  - helix-network

nginx:
  image: nginx:alpine
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./docker/nginx.conf:/etc/nginx/nginx.conf:ro
  depends_on:
    - helixcode-server
  networks:
    - helix-network

volumes:
  postgres_data:
  redis_data:

networks:
  helix-network:
    driver: bridge
```

9.2 Health Check Requirements

Per CONST-019: Container Up Healthy. Each service MUST implement deep health checks:

Service	Health Check	Deep Check
helixcode-server	GET /health	DB connection, provider availability
postgres	pg_isready	SELECT 1
redis	redis-cli ping	PONG
nginx	curl localhost	upstream server response

10. Deployment Architecture

10.1 Development

```
Developer Machine
|
+---> make dev
|
```

```
+---> config/dev/config.yaml
+---> PostgreSQL (local or container)
+---> Redis (local or container)
+---> ./bin/helixcode (server)
```

10.2 Staging

```
Docker Compose Stack
|
+---> nginx (reverse proxy)
+---> helixcode-server (2 replicas)
+---> PostgreSQL (1 instance)
+---> Redis (1 instance + sentinel)
+---> Prometheus + Grafana
```

10.3 Production (Kubernetes)

```
K8s Cluster
|
+---> Ingress (nginx-ingress)
+---> helixcode-deployment (3+ replicas, HPA)
+---> PostgreSQL StatefulSet
+---> Redis Cluster
+---> Prometheus + Grafana
+---> cert-manager (TLS)
```

11. Monitoring & Observability

11.1 Metrics (Prometheus)

Metric	Type	Description
helix_requests_total	Counter	Total HTTP requests
helix_request_duration_seconds	Histogram	Request latency
helix_llm_generation_duration	Histogram	LLM generation time
helix_llm_provider_failures	Counter	Provider failure count
helix_workers_total	Gauge	Active workers
helix_tasks_pending	Gauge	Pending tasks
helix_db_connections	Gauge	DB connection pool

11.2 Logging (Structured)

```
{
  "timestamp": "2026-04-30T12:00:00Z",
  "level": "info",
  "service": "helixcode",
  "component": "llm",
  "provider": "ollama",
```

```
"model": "llama3.2",
"request_id": "uuid",
"duration_ms": 1250,
"tokens_input": 10,
"tokens_output": 50,
"status": "success"
}
```

11.3 Tracing (OpenTelemetry)

- Request tracing across API -> LLM -> Provider
- Task execution tracing across Task -> Worker -> Result
- Error tracing with full stack context

12. Multi-Platform Architecture

12.1 Platform Matrix

Platform	UI Framework	Build Target	Status
Linux	CLI + TUI	x86_64, ARM64	Target
macOS	CLI + TUI + Desktop	x86_64, ARM64	Target
Windows	CLI + Desktop	x86_64	Target
iOS	SwiftUI	.xcframework	Target
Android	Jetpack Compose	.aar	Target
Aurora OS	Qt/QML	RPM	Target
Harmony OS	ArkUI	.hap	Target

12.2 Shared Core

```
shared/mobile-core/
|
+---> Go bindings (gomobile bind)
    |
    +---> iOS: .xcframework
    +---> Android: .aar
```

Appendix A: File References

Component	Key Files	Lines (Approx)
Auth	internal/auth/auth.go	470
CLI	cmd/cli/main.go	341
Dockerfile	Dockerfile	55
go.mod	go.mod	10
.gitmodules	.gitmodules	233

Appendix B: Verification Commands

```
# Verify architecture diagrams are accurate
cat internal/auth/auth.go | grep -c "func.*AuthService" # Should be > 0
cat cmd/cli/main.go | grep -c "simulate" # Should be 0 after fix

# Verify database schema
cat internal/database/schema.sql | grep "CREATE TABLE" | wc -l # Should be >= 11

# Verify API endpoints
cat internal/server/routes.go | grep "POST\|GET\|PUT\|DELETE" | wc -l # Should be >= 20

# Verify provider implementations
ls internal/llm/providers/ | wc -l # Should be >= 15

# Verify health checks
cat docker-compose.yml | grep -c "healthcheck" # Should be >= 4
```